

LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK

PERTEMUAN 2
Unit Testing Life Cycle



Disusun Oleh:

Nama : Abdullah Afif Habiburrohman
NIM : 537611/SV/24441
Kelas : BB / B2
Dosen : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

Contents

A Tujuan Pembelajaran	2
B Dasar Teori	2
C Hasil dan Pembahasan	3
C.1 Kode	3
C.2 Gambar	7
C.3 GitHub	8
D Kesimpulan	8
E Daftar Pustaka	8

A. Tujuan Pembelajaran

1. Mahasiswa memahami life cycle dan anotasi yang ada pada JUnit
2. Mahasiswa mampu mengimplementasikan anotasi pada pengujian unit

B. Dasar Teori

Dalam kerangka kerja pengujian Java (khususnya JUnit versi 5 ke atas), siklus hidup pengujian (test lifecycle) dikelola melalui serangkaian anotasi khusus. Siklus hidup ini bertujuan untuk mengatur tiga fase utama dalam setiap skenario pengujian, yaitu: fase persiapan (setup), fase eksekusi pengujian, dan fase pembersihan (cleanup). Pengelolaan siklus hidup ini sangat penting untuk menyiapkan lingkungan pengujian (test environment) yang konsisten, memastikan keandalan tes, serta mencegah terjadinya kebocoran sumber daya (resource leaks).

Lima anotasi fundamental yang menyusun test lifecycle pada JUnit adalah:

1. `@BeforeAll`: Menandai metode yang akan dieksekusi satu kali sebelum seluruh metode tes di dalam sebuah kelas pengujian dijalankan. Biasanya digunakan untuk inisialisasi pada tingkat kelas, seperti membuka koneksi database atau menyiapkan sumber daya yang digunakan bersama (shared resources).
2. `@BeforeEach`: Menandai metode yang akan dieksekusi sesaat sebelum setiap metode tes berjalan. Fase ini ideal untuk menyiapkan data mock, mereset variabel, atau menginisialisasi objek agar setiap metode tes mendapatkan lingkungan yang bersih dan independen.
3. `@Test`: Menandai metode utama yang berisi logika pengujian dan asersi (assertions) untuk memvalidasi apakah kode yang diuji berjalan sesuai dengan ekspektasi.
4. `@AfterEach`: Menandai metode yang dieksekusi setelah setiap metode tes selesai dijalankan. Berguna untuk membersihkan data spesifik atau mereset konfigurasi pasca-pengujian.
5. `@AfterAll`: Menandai metode yang dieksekusi satu kali setelah seluruh metode tes dalam sebuah kelas selesai. Fungsi utamanya adalah untuk melepaskan sumber daya memori atau menutup koneksi tingkat kelas.

Secara bawaan (default), JUnit menggunakan strategi siklus hidup `PER_METHOD`, di mana JUnit akan membuat instance objek kelas pengujian baru untuk setiap metode `@Test`. Meskipun hal ini menjamin isolasi total antar tes, pendekatan ini mengharuskan metode `@BeforeAll` dan `@AfterAll` bersifat static.

Untuk mengatasi batasan tersebut atau untuk mendukung pengujian yang memerlukan pembagian status (shared state), JUnit menyediakan anotasi `@TestInstance(TestInstance.Lifecycle.PER_CLASS)`. Saat anotasi ini diterapkan pada tingkat kelas, JUnit hanya akan membuat satu instance kelas pengujian yang digunakan

oleh seluruh metode tes di dalamnya. Pendekatan ini memungkinkan metode inisialisasi tingkat kelas berjalan tanpa keyword static dan lebih efisien dalam manajemen memori untuk kelas uji yang berat secara komputasi.

Menurut prinsip standar unit testing, setiap metode tes idealnya bersifat independen dan tidak bergantung pada urutan eksekusi. Oleh karena itu, secara default, JUnit mengeksekusi tes menggunakan algoritma deterministik yang disengaja agar urutannya tidak terduga (nonobvious).

Namun, untuk skenario tertentu seperti integration test atau pengujian fungsional yang mempertahankan status (stateful testing—terutama saat menggunakan PER_CLASS), urutan eksekusi menjadi sangat krusial. JUnit menyediakan mekanisme untuk mengontrol urutan eksekusi ini melalui:

1. `@TestMethodOrder`: Anotasi pada tingkat kelas untuk menentukan strategi pengurutan metode. Salah satu strategi bawaan adalah `MethodOrderer.OrderAnnotation.class`, yang memerintahkan JUnit untuk mengeksekusi tes berdasarkan nilai numerik yang didefinisikan secara eksplisit.
2. `@Order(n)`: Anotasi pada tingkat metode yang bekerja berpasangan dengan strategi `OrderAnnotation`. Metode akan dieksekusi secara berurutan mulai dari nilai numerik terkecil.
3. `@TestClassOrder`: Digunakan untuk mengurutkan eksekusi antar kelas pengujian, khususnya pada nested test classes (kelas bersarang), menggunakan strategi seperti `ClassOrderer.OrderAnnotation.class`.

C. Hasil dan Pembahasan

Mahasiswa melanjutkan praktikum sebelumnya, Wallet dan testing, dengan menambahkan Testing Lifecycle pada kelas `WalletTest`. Mahasiswa akan menampilkan kode tambahan/perubahan yang dilakukan pada kelas `WalletTest` untuk mengimplementasikan Testing Lifecycle. Berikut adalah kode lengkap dari kelas `WalletTest` yang telah diimplementasikan dengan Testing Lifecycle:

C.1 Kode

```
1 package org.example;  
2  
3 import org.junit.jupiter.api.*;  
4  
5 import java.util.List;  
6  
7 import static org.junit.jupiter.api.Assertions.*;
```

Kode di atas adalah kode yang digunakan untuk mengimpor kelas-kelas yang diperlukan untuk melakukan pengujian pada kelas `Wallet`.

```
1 @TestInstance(TestInstance.Lifecycle.PER_CLASS)  
2 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
```

Kode di atas adalah kode yang digunakan untuk mengatur lifecycle dari kelas pengujian dan urutan eksekusi metode pengujian. `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` digunakan untuk mengatur lifecycle dari kelas pengujian menjadi `PER_CLASS`, yang berarti objek pengujian akan dibuat sekali untuk seluruh metode pengujian dalam kelas. Hal ini membuat `@BeforeAll` dan `@AfterAll` dapat digunakan pada metode non-static.

`@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` digunakan untuk mengatur urutan eksekusi metode pengujian berdasarkan anotasi `@Order`. Selain `OrderAnnotation`, terdapat beberapa strategi lain untuk mengatur urutan eksekusi metode pengujian, seperti `Alphanumeric`, `Random`, dan `MethodName`. `Alphanumeric` akan mengurutkan metode pengujian berdasarkan nama metode secara alfabetis, `Random` akan mengacak urutan eksekusi metode pengujian setiap kali dijalankan, dan `MethodName` akan mengurutkan metode pengujian berdasarkan nama metode secara berurutan. Dengan menggunakan `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`, kita dapat menentukan urutan eksekusi metode pengujian secara eksplisit dengan memberikan anotasi `@Order` pada setiap metode pengujian.

```
1 class WalletTest {
2
3     Wallet dompet;
4
5     @BeforeAll
6     void initClass() {
7         dompet = new Wallet("Abdullah");
8     }
9
10    @AfterAll
11    void cleanClass() {
12        dompet = null;
13    }
14
15    @BeforeEach
16    void initMethod() {
17        dompet.addMoney(10000);
18        dompet.addCard("DefaultBank", "000-000");
19    }
20
21    @AfterEach
22    void cleanMethod() {
23        dompet.cleanCards();
24        dompet.cleanMoneys();
25    }
```

Kode di atas adalah kode yang digunakan untuk mengatur lifecycle dari metode pengujian. `@BeforeAll` digunakan untuk menandai metode yang akan dijalankan sekali sebelum semua metode pengujian dalam kelas dijalankan. Dalam contoh ini, metode `initClass()` digunakan untuk menginisialisasi objek `dompet` sebelum semua metode pengujian dijalankan. `@AfterAll` digunakan untuk menandai metode yang akan dijalankan sekali setelah semua metode pengujian dalam kelas dijalankan. Dalam contoh ini, metode `cleanClass()` digunakan untuk membersihkan objek `dom-`

pet setelah semua metode pengujian dijalankan.

@BeforeEach digunakan untuk menandai metode yang akan dijalankan sebelum setiap metode pengujian dijalankan. Dalam contoh ini, metode `initMethod()` digunakan untuk menambahkan uang dan kartu default ke dalam dompet sebelum setiap metode pengujian dijalankan. @AfterEach digunakan untuk menandai metode yang akan dijalankan setelah setiap metode pengujian dijalankan. Dalam contoh ini, metode `cleanMethod()` digunakan untuk membersihkan kartu dan uang dari dompet setelah setiap metode pengujian dijalankan. Dengan menggunakan anotasi-anotasi ini, kita dapat memastikan bahwa setiap metode pengujian dijalankan dalam kondisi yang konsisten dan terisolasi, sehingga hasil pengujian menjadi lebih akurat dan dapat dipercaya.

```
1  @Test
2  @Order(1)
3  @DisplayName("Test Inisialisasi Wallet dan Owner")
4  void testIsWallet() {
5      assertNotNull(dompot, "Objek dompet tidak boleh null");
6      assertEquals("Abdullah", dompot.getOwner());
7  }
8
9  @Test
10 @Order(2)
11 @DisplayName("Test Setter Owner")
12 void testOwner() {
13     dompot.setOwner("Afif");
14     assertEquals("Afif", dompot.getOwner());
15 }
```

Pada kode di atas, terdapat dua metode pengujian yang menggunakan anotasi @Test, @Order, dan @DisplayName. Metode pertama, `testIsWallet()`, digunakan untuk menguji apakah objek dompet telah diinisialisasi dengan benar dan apakah pemilik dompet sesuai dengan yang diharapkan. Metode kedua, `testOwner()`, digunakan untuk menguji apakah metode setter dan getter untuk properti owner bekerja dengan benar. Dengan menggunakan anotasi @Order, kita dapat mengatur urutan eksekusi metode pengujian. Dalam contoh ini, metode `testIsWallet()` akan dijalankan sebelum metode `testOwner()`. Dengan menggunakan anotasi @DisplayName, kita dapat memberikan nama yang lebih deskriptif dan mudah dipahami untuk setiap metode pengujian. Hal ini dapat membantu dalam memahami tujuan dari setiap metode pengujian dan mempermudah proses debugging jika terjadi kegagalan pengujian.

```
1  @Test
2  @DisplayName("Test Menambah dan Mengambil Kartu")
3  void testCardOperations() {
4      dompot.addCard("BCA", "111-222");
5      dompot.addCard("Mandiri", "333-444");
6
7      assertEquals(3, dompot.getCards().size());
8
9      Card diambil = dompot.takeCard("111-222");
10 }
```

```

11         assertEquals("Verifikasi Kartu",
12             () -> assertNotNull(diambil),
13             () -> assertEquals("BCA", diambil.getNamaBank())
14     ),
15     () -> assertEquals("111-222", diambil.
16     getNomorRekening())
17 );
18
19     assertEquals(2, dompet.getCards().size());
20 }
21
22 @Test
23 @DisplayName("Test Tambah Uang dan Hitung Total")
24 void testAddMoney() {
25     assertEquals(10000, dompet.calculateTotalBalance());
26
27     dompet.addMoney(5000);
28     dompet.addMoney(2000);
29
30     assertEquals(3, dompet.getMoneys().size());
31     assertEquals(
32         List.of(10000, 5000, 2000),
33         dompet.getMoneys()
34     );
35
36     assertEquals(17000, dompet.calculateTotalBalance());
37 }
38
39 @Test
40 @DisplayName("Test Mengambil Uang (Withdraw)")
41 void testTakeMoney() {
42     dompet.addMoney(50000);
43     dompet.addMoney(20000);
44
45     boolean isSuccess = dompet.takeMoney(20000);
46
47     assertTrue(isSuccess, "Harusnya berhasil mengambil uang
48     yang tersedia");
49     assertEquals(60000, dompet.calculateTotalBalance(),
50         "Sisa saldo harus 60.000");
51
52     boolean isFail = dompet.takeMoney(100000);
53
54     assertFalse(isFail, "Harusnya gagal mengambil uang yang
55     tidak ada");
56     assertEquals(60000, dompet.calculateTotalBalance(),
57         "Saldo tidak boleh berubah jika gagal");
58 }
59
60 @Test
61 @DisplayName("Test Mengambil Kartu Yang Tidak Ada")
62 void testTakeCardNotFound() {
63     Card result = dompet.takeCard("999-999");
64
65     assertNull(result);
66 }

```

```

63         assertEquals(1, dompet.getCards().size());
64     }
65
66     @Test
67     @DisplayName("Test Tambah Uang Negatif")
68     void testAddNegativeMoney() {
69         dompet.addMoney(-5000);
70
71         assertEquals(1, dompet.getMoneys().size());
72         assertEquals(10000, dompet.calculateTotalBalance());
73     }
74
75     @Test
76     @DisplayName("Test Ambil Uang Saat Kosong")
77     void testTakeMoneyWhenEmpty() {
78         dompet.cleanMoneys();
79
80         boolean result = dompet.takeMoney(10000);
81
82         assertFalse(result);
83         assertEquals(0, dompet.calculateTotalBalance());
84     }
85 }

```

Kode di atas adalah kode pengembangan dari praktikum sebelumnya. Pengembangan yang dilakukan difokuskan untuk penyesuaian dan pengimplementasian dari Testing Lifecycle. Misal pada modul `testCardOperations()`, diubah nilai `assertEquals(2, dompet.getCards().size())` menjadi `assertEquals(3, dompet.getCards().size())` karena pada metode `initMethod()` sudah ditambahkan 1 kartu default sebelum setiap metode pengujian dijalankan. Untuk modul-modul lain juga sama terdapat penyesuaian.

C.2 Gambar

Berikut adalah hasil run dari kelas `WalletTest` yang telah diimplementasikan dengan Testing Lifecycle:

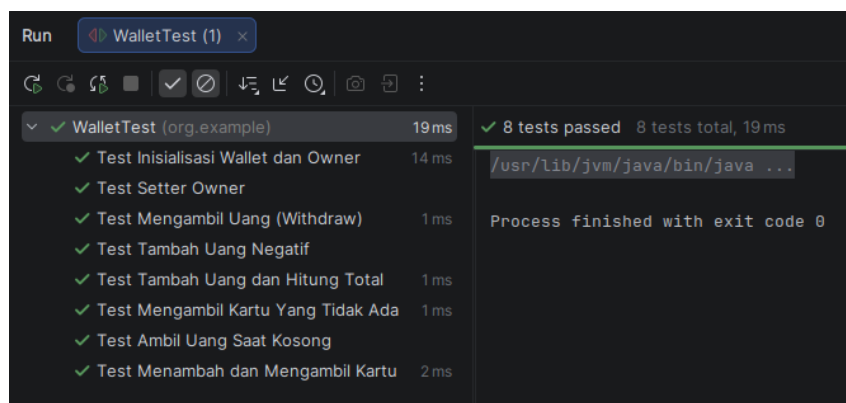


Figure 1: Hasil Run Kelas `WalletTest` dengan Testing Lifecycle

C.3 GitHub

Berikut adalah link repository di GitHub yang berisi kode diatas <https://github.com/bukunya/PPPL/tree/Pertemuan2>

D. Kesimpulan

Dalam praktikum ini mahasiswa telah mempelajari siklus hidup pengujian di JUnit 5 dan penerapan anotasi seperti @BeforeAll, @BeforeEach, @AfterEach, dan @AfterAll. Dengan mengatur 'TestInstance' menjadi PER_CLASS dan menggunakan '@TestMethodOrder', pengujian menjadi lebih terstruktur dan memungkinkan penggunaan status bersama. Penambahan metode inisialisasi dan pembersihan memastikan setiap test berjalan dalam kondisi yang konsisten.

E. Daftar Pustaka

- [1] JUnit 5 Test Lifecycle. <https://www.geeksforgeeks.org/software-testing/junit-5-test-lifecycle/>
- [2] TestInstance Annotation di JUnit. <https://www.baeldung.com/junit-testinstance-annotation>
- [3] JUnit 5 User Guide. <https://docs.junit.org/5.10.2/user-guide/index.html>